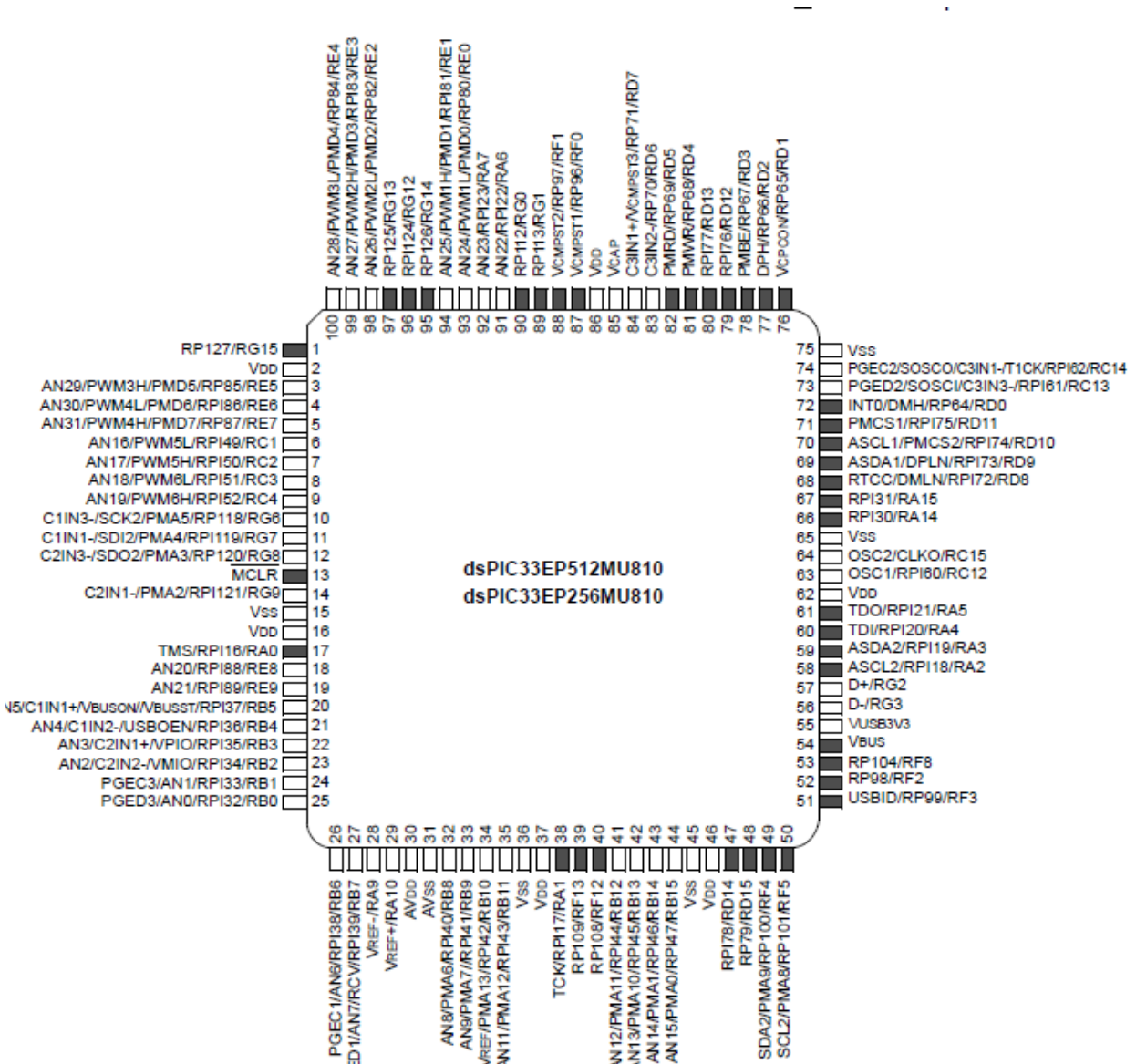
Università di Genova

Embedded Systems

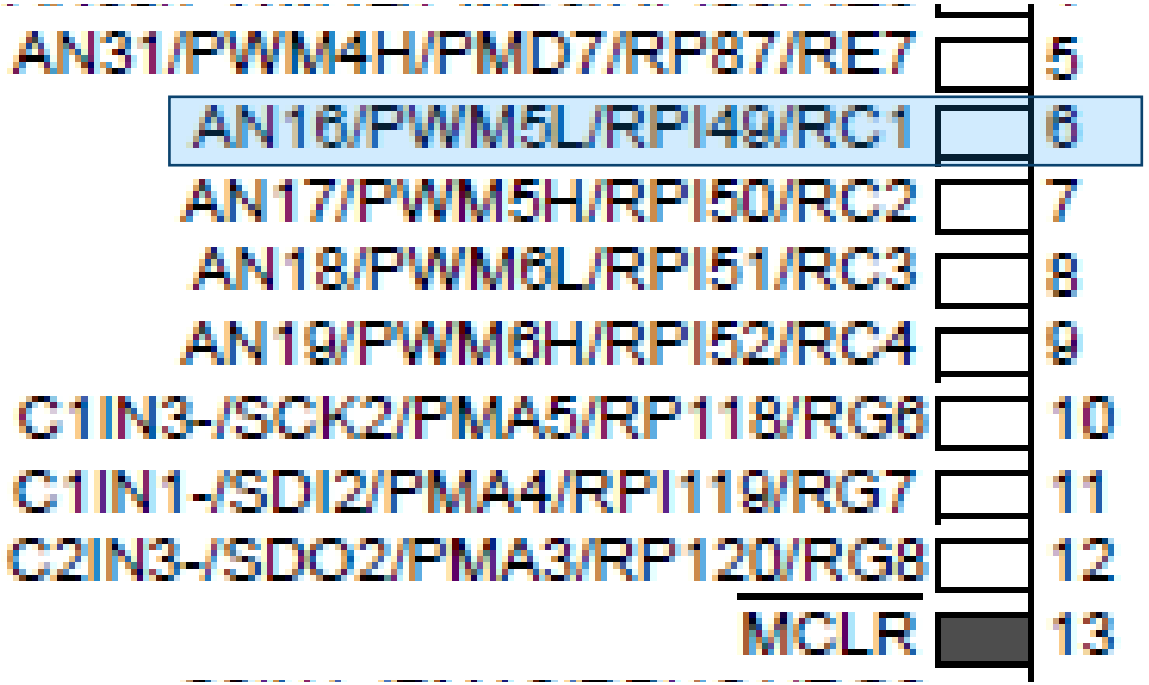
Digital I/O

Prof. Enrico Simetti

I/O Pins

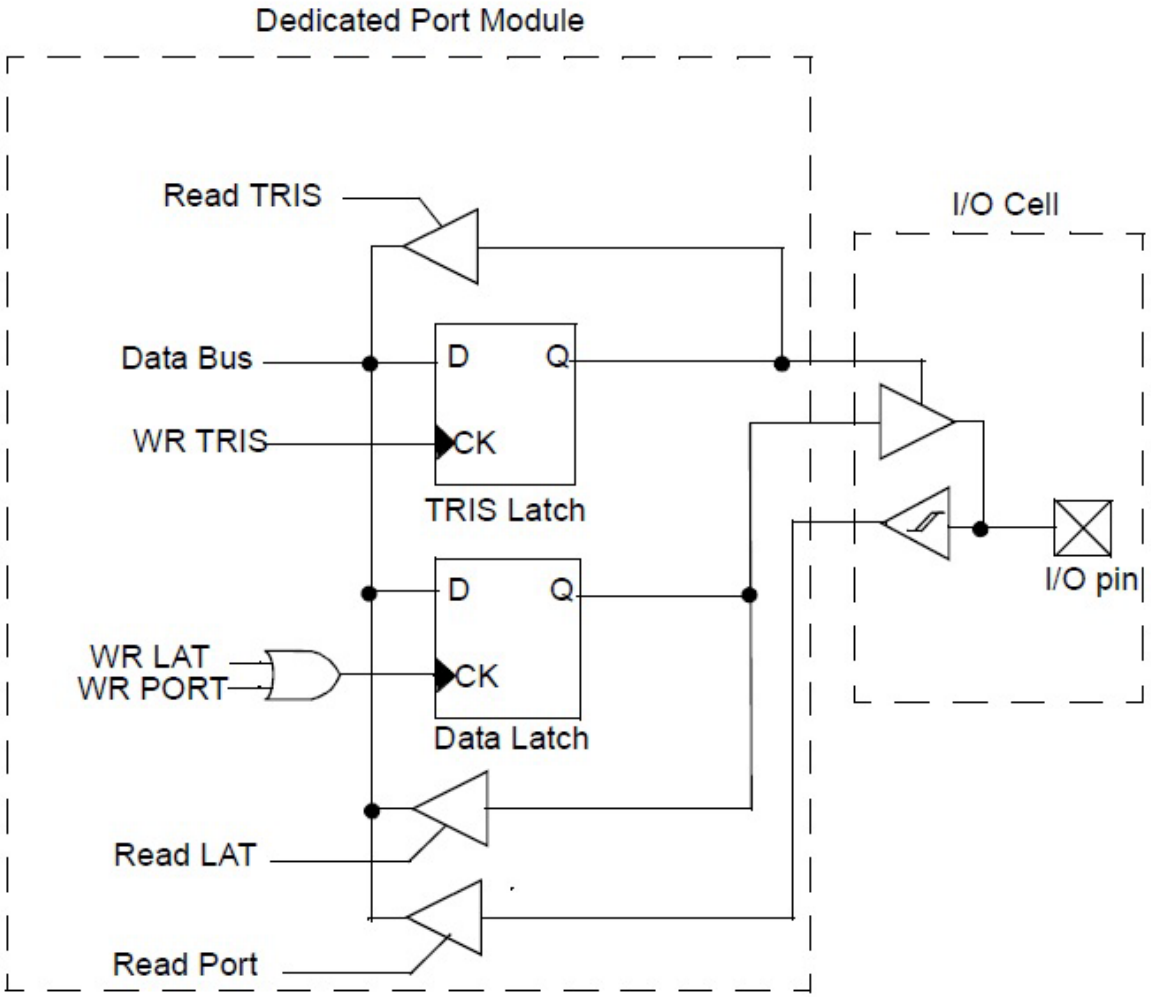

dsPIC33EP512MU810
dsPIC33EP256MU810
100 AN28/PWM3L/PMD4/RP84/RE4
99 AN27/PWM2H/PMD3/RPI83/RE3
98 AN26/PWM2L/PMD2/RP82/RE2
97 RP125/RG13
96 RPI124/RG12
95 RP126/RG14
94 AN25/PWM1H/PMD1/RPI81/RE1
93 AN24/PWM1L/PMD0/RP80/RE0
92 AN23/RPI123/RA7
91 AN22/RPI122/RA6
90 RP112/RG0
89 RP113/RG1
88 VCMPST2/RP97/RF1
87 VCMPST1/RP96/RF0
86 VDD
85 VCAP
84 C3IN1+/VCMPST3/RP71/RD7
83 C3IN2-/RP70/RD6
82 PMRD/RP69/RD5
81 PMWR/RP68/RD4
80 RPI77/RD13
79 RPI76/RD12
78 PMBE/RP67/RD3
77 DPH/RP66/RD2
76 VCPCON/RP65/RD1
1 RP127/RG15
2 VDD
3 AN29/PWM3H/PMD5/RP85/RE5
4 AN30/PWM4L/PMD6/RPI86/RE6
5 AN31/PWM4H/PMD7/RP87/RE7
6 AN16/PWM5L/RPI49/RC1
7 AN17/PWM5H/RPI50/RC2
8 AN18/PWM6L/RPI51/RC3
9 AN19/PWM6H/RPI52/RC4
10 C1IN3-/SCK2/PMA5/RP118/RG6
11 C1IN1-/SDI2/PMA4/RPI119/RG7
12 C2IN3-/SDO2/PMA3/RP120/RG8
13 MCLR
14 C2IN1-/PMA2/RPI121/RG9
15 VSS
16 VDD
17 TMS/RPI16/RA0
18 AN20/RPI88/RE8
19 AN21/RPI89/RE9
20 N5/C1IN1+/VBUSON/VBUSST/RPI37/RB5
21 AN4/C1IN2-/USBOEN/RPI36/RB4
22 AN3/C2IN1+/VPIO/RPI35/RB3
23 AN2/C2IN2-/VMIO/RPI34/RB2
24 PGEC3/AN1/RPI33/RB1
25 PGED3/AN0/RPI32/RB0
75 VSS
74 PGEC2/SOSCO/C3IN1-/T1CK/RPI62/RC14
73 PGED2/SOSCI/C3IN3-/RPI61/RC13
72 INT0/DMH/RP64/RD0
71 PMCS1/RPI75/RD11
70 ASCL1/PMCS2/RPI74/RD10
69 ASDA1/DPLN/RPI73/RD9
68 RTCC/DMLN/RPI72/RD8
67 RPI31/RA15
66 RPI30/RA14
65 VSS
64 OSC2/CLKO/RC15
63 OSC1/RPI60/RC12
62 VDD
61 TDO/RPI21/RA5
60 TDI/RPI20/RA4
59 ASDA2/RPI19/RA3
58 ASCL2/RPI18/RA2
57 D+/RG2
56 D-/RG3
55 VUSB3V3
54 VBUS
53 RP104/RF8
52 RP98/RF2
51 USBID/RP99/RF3
26 PGEC1/AN6/RPI38/RB6
27 PGED1/AN7/RCV/RPI39/RB7
28 VREF-/RA9
29 VREF+/RA10
30 AVDD
31 AVSS
32 AN8/PMA6/RPI40/RB8
33 AN9/PMA7/RPI41/RB9
34 AN10/CVREF/PMA13/RPI42/RB10
35 AN11/PMA12/RPI43/RB11
36 VSS
37 VDD
38 TCK/RPI17/RA1
39 RP109/RF13
40 RP108/RF12
41 AN12/PMA11/RPI44/RB12
42 AN13/PMA10/RPI45/RB13
43 AN14/PMA1/RPI46/RB14
44 AN15/PMA0/RPI47/RB15
45 VSS
46 VDD
47 RPI78/RD14
48 RP79/RD15
49 SDA2/PMA9/RP100/RF4
50 SCL2/PMA8/RP101/RF5

I/O Pins



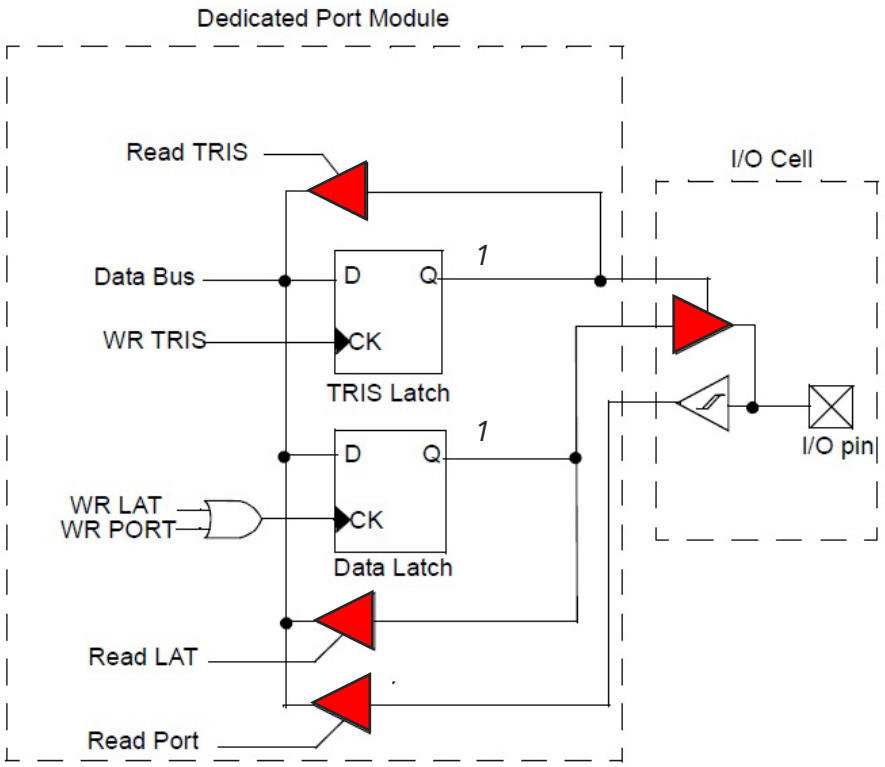
PIN 6 is shared between AN16, PWM5L, RPI49 and RC1
The priority goes from left to right, so AN16 has more priority than PWM5L and so on.

Digital I/O Port



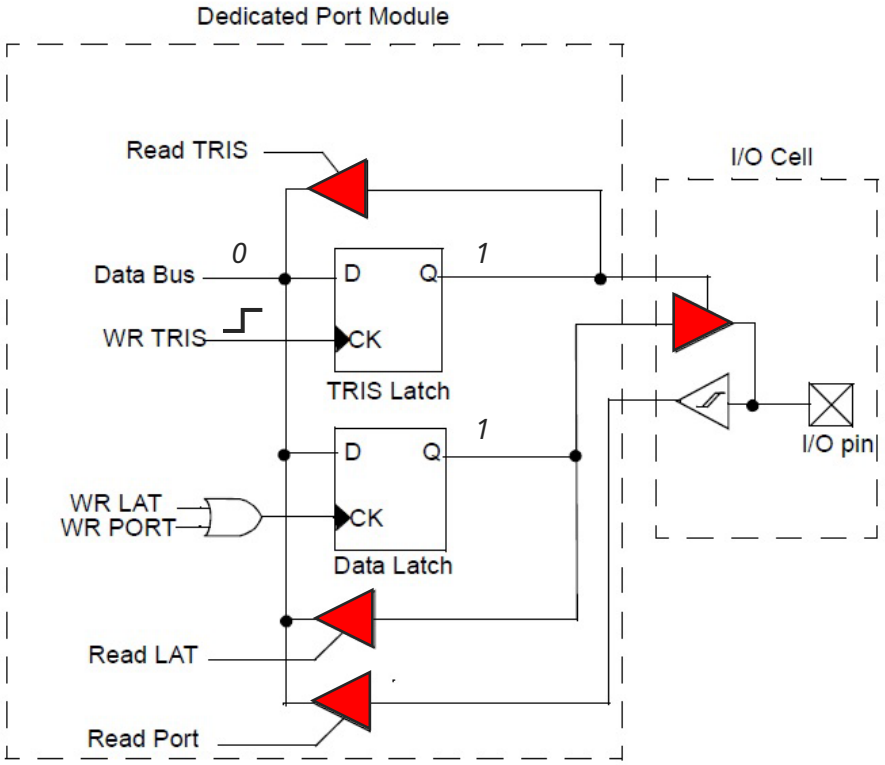
Digital I/O Port

Suppose we want to write to the pin



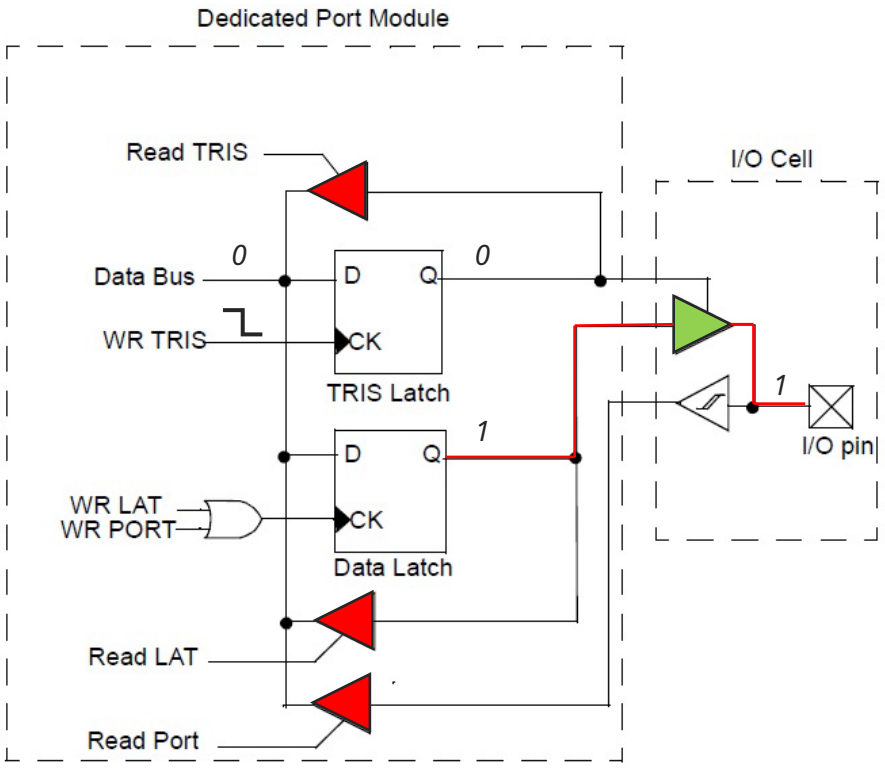
Digital I/O Port

Suppose we want to write to the pin
We issue the instruction WR TRIS, 0



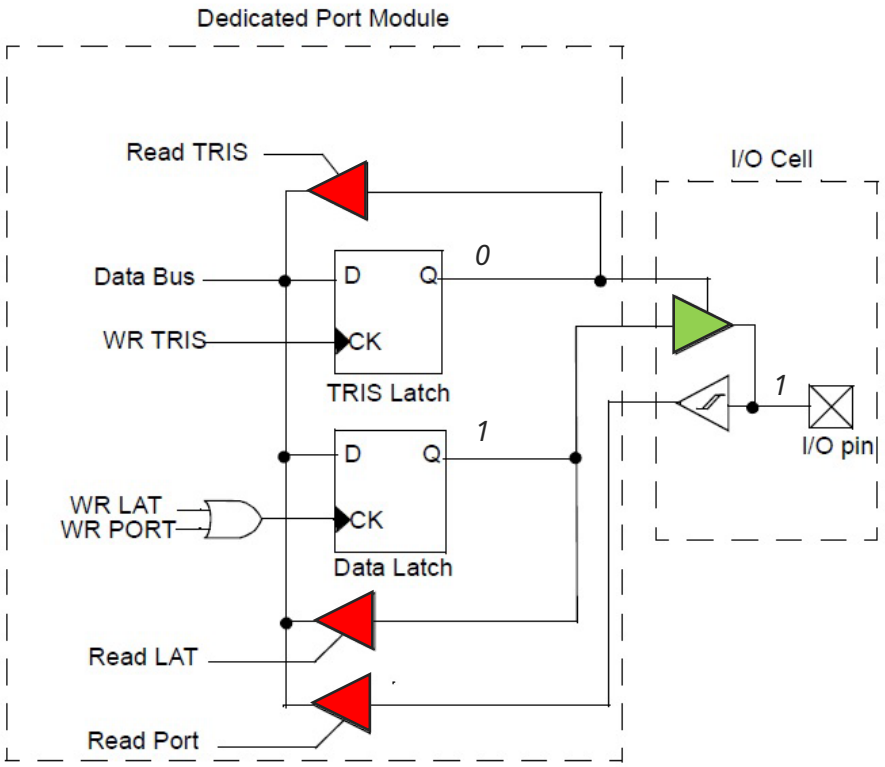
Digital I/O Port

At the end, the data held in the DATA Latch is available on the pin



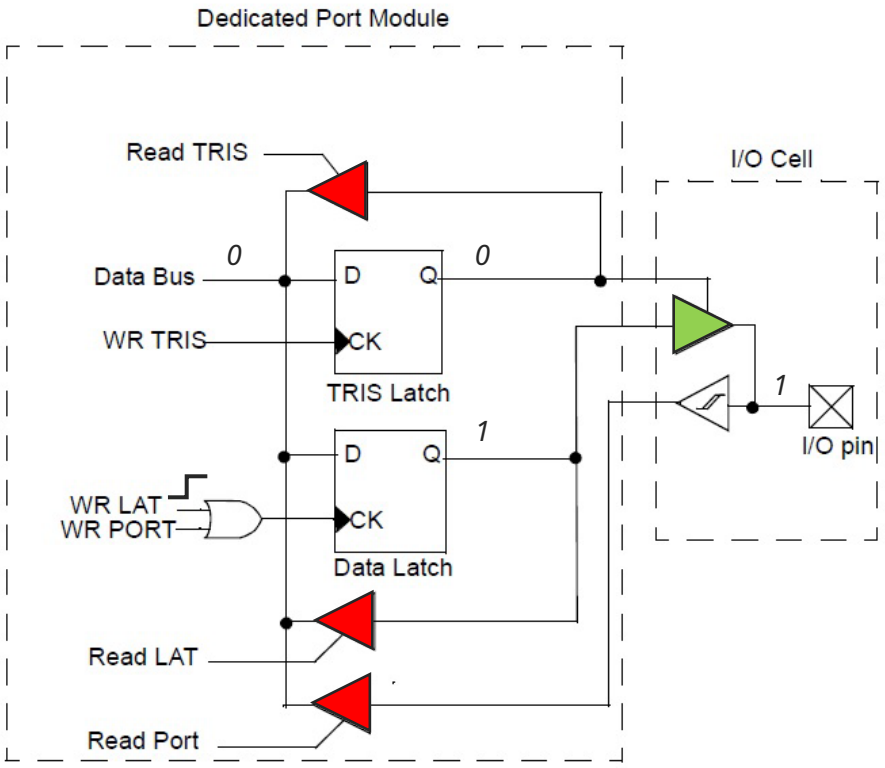
Digital I/O Port

Let's now change the data to 0
Let's issue the instruction WR LAT, 0



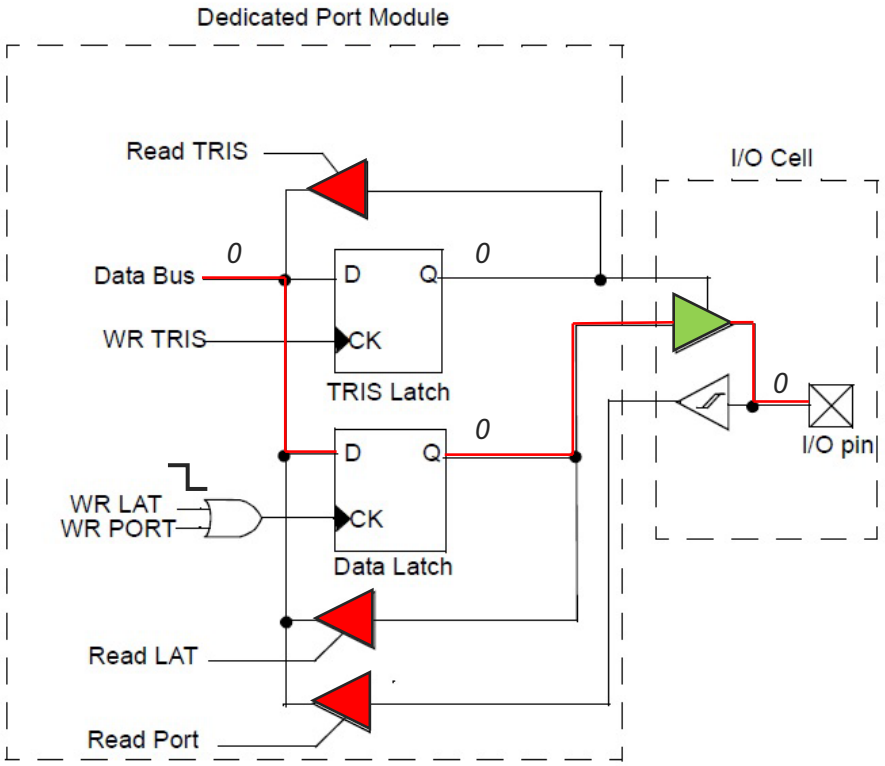
Digital I/O Port

Let's now change the data to 0
Let's issue the instruction WR LAT, 0



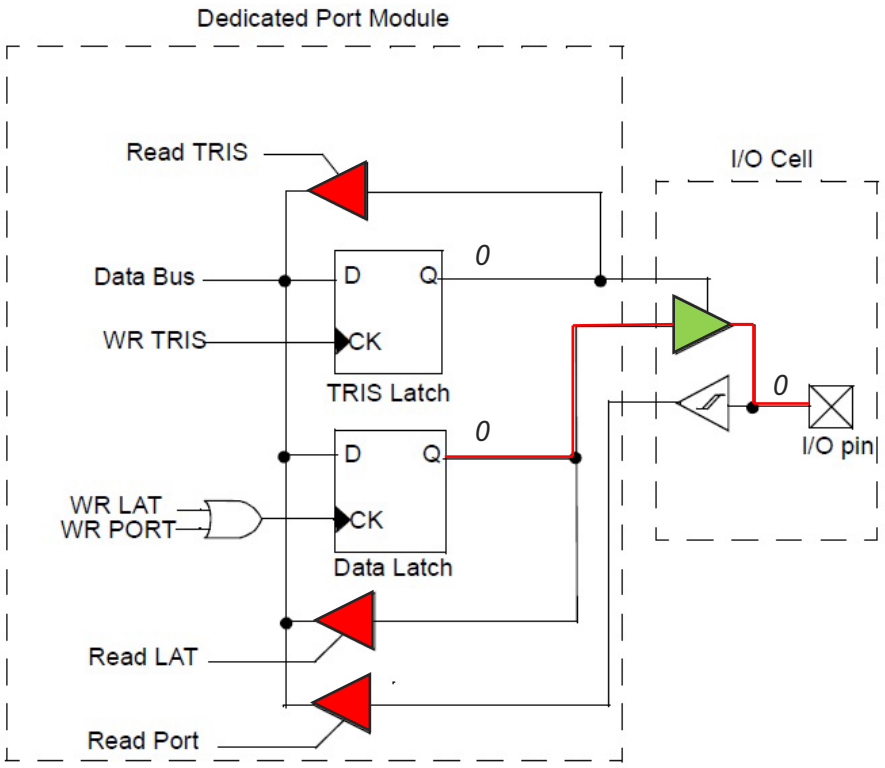
Digital I/O Port

Let's now change the data to 0
Let's issue the instruction WR LAT, 0



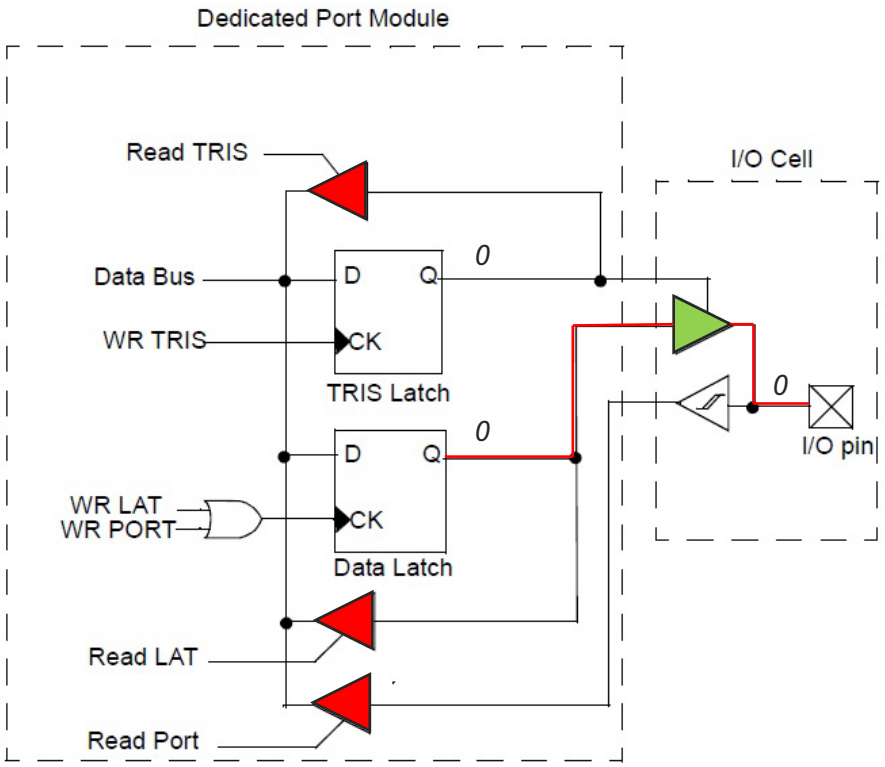
Digital I/O Port

When the instruction is done, the DATA Latch (memory element) keeps the pin to 0



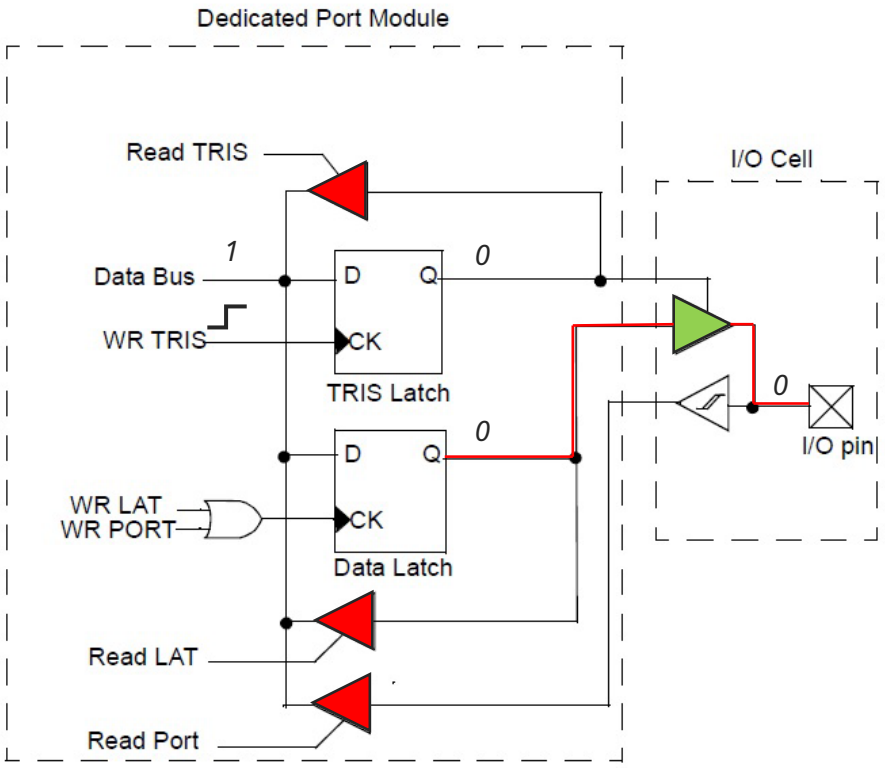
Digital I/O Port

How to read the PIN?
First, let's reconfigure to input by issuing the instruction WR TRIS, 1



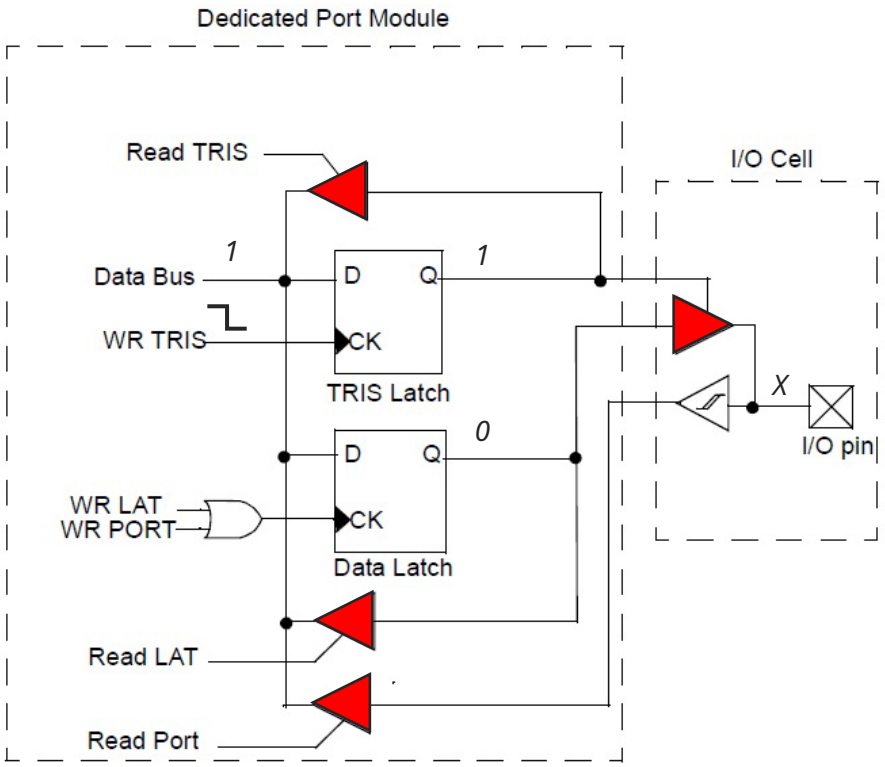
Digital I/O Port

How to read the PIN?
First, let's reconfigure to input by issuing the instruction WR TRIS, 1



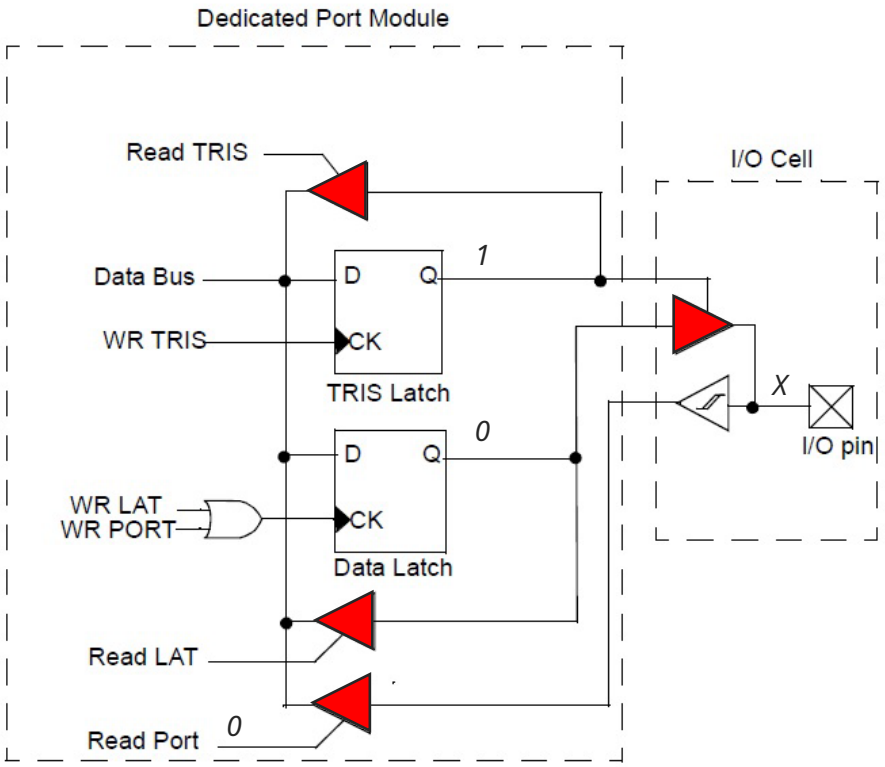
Digital I/O Port

How to read the PIN?
First, let's reconfigure to input by issuing the instruction WR TRIS, 1



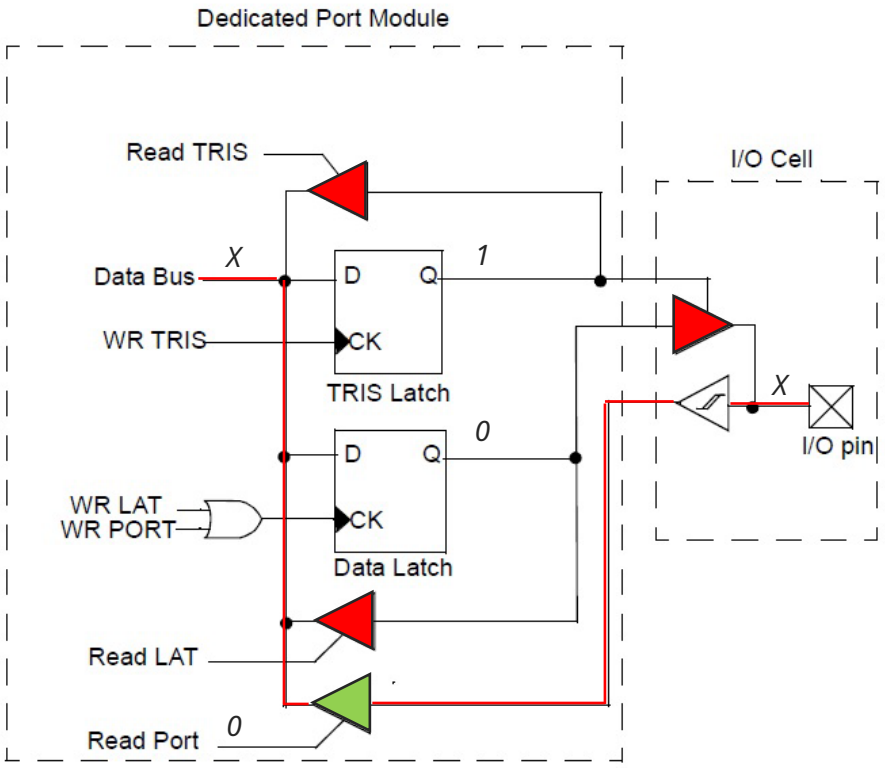
Digital I/O Port

Then, we read using the Read Port instruction



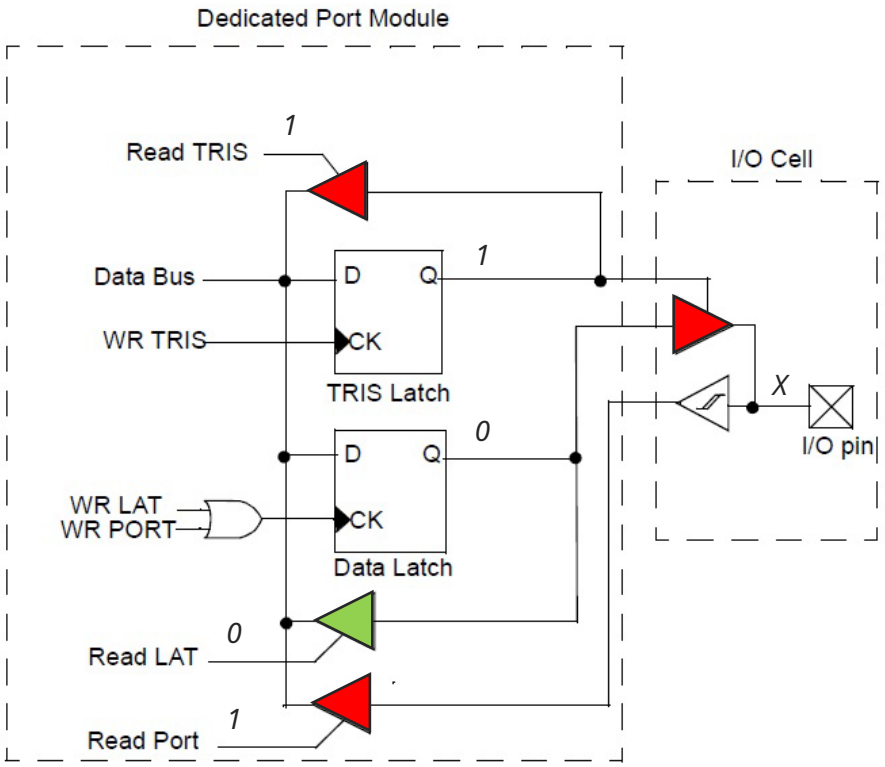
Digital I/O Port

And now the value X (0 or 1) on the pin is available to the CPU through the data bus



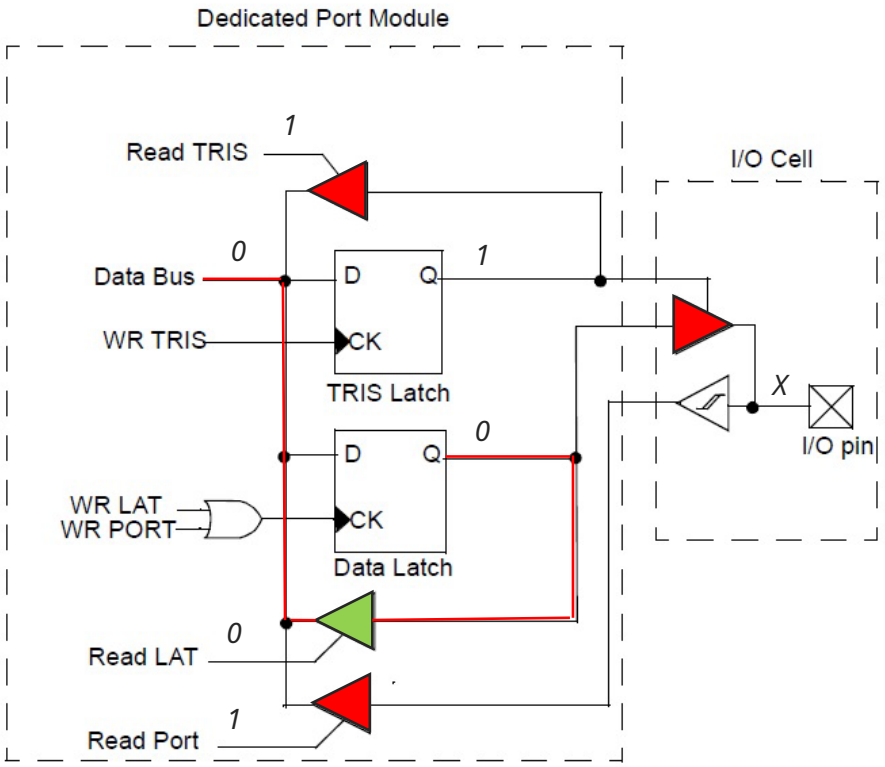
Digital I/O Port

What happens if we use Read Lat (reading the data latch)?

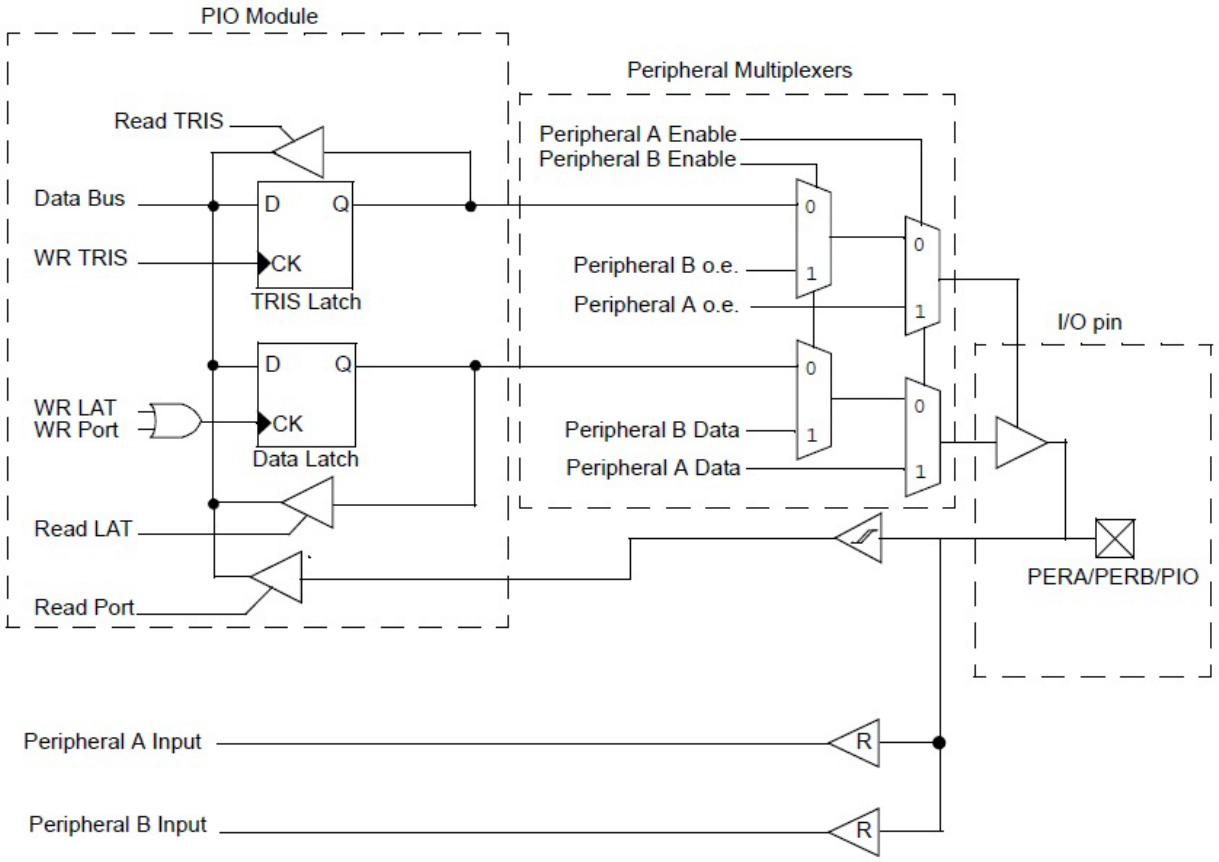


Digital I/O Port

The tristate button closes and the Q value of the Data Latch is available on the bus for the CPU to read

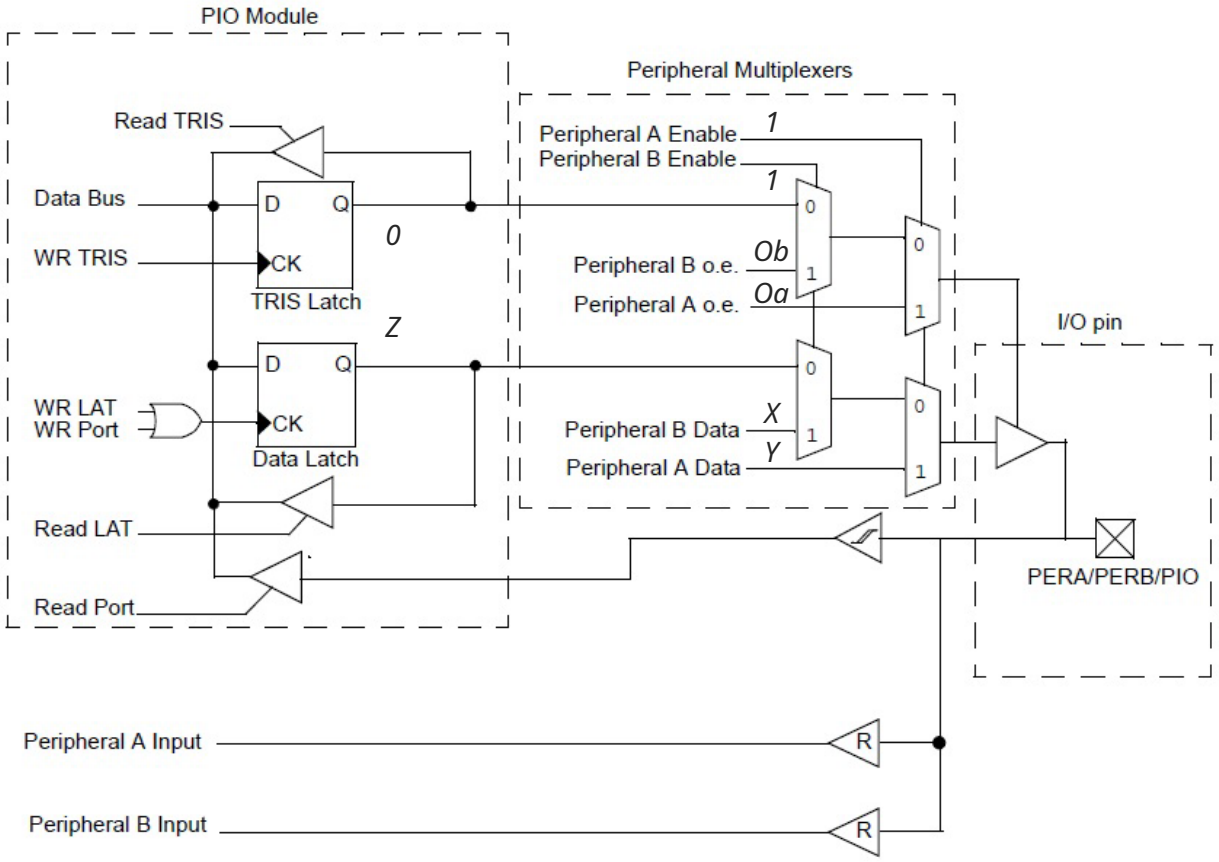


Multiplexed I/O Pins



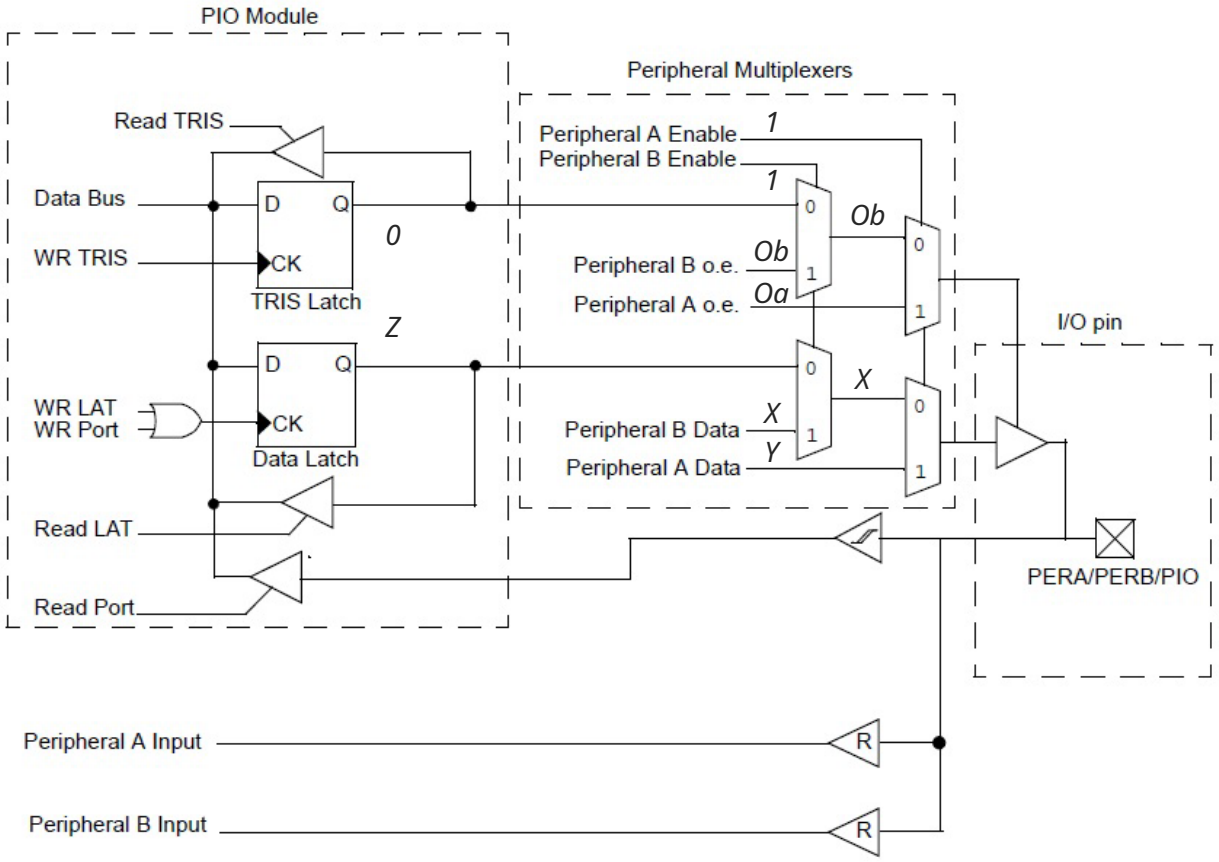
Multiplexed I/O Pins

Initial conditions



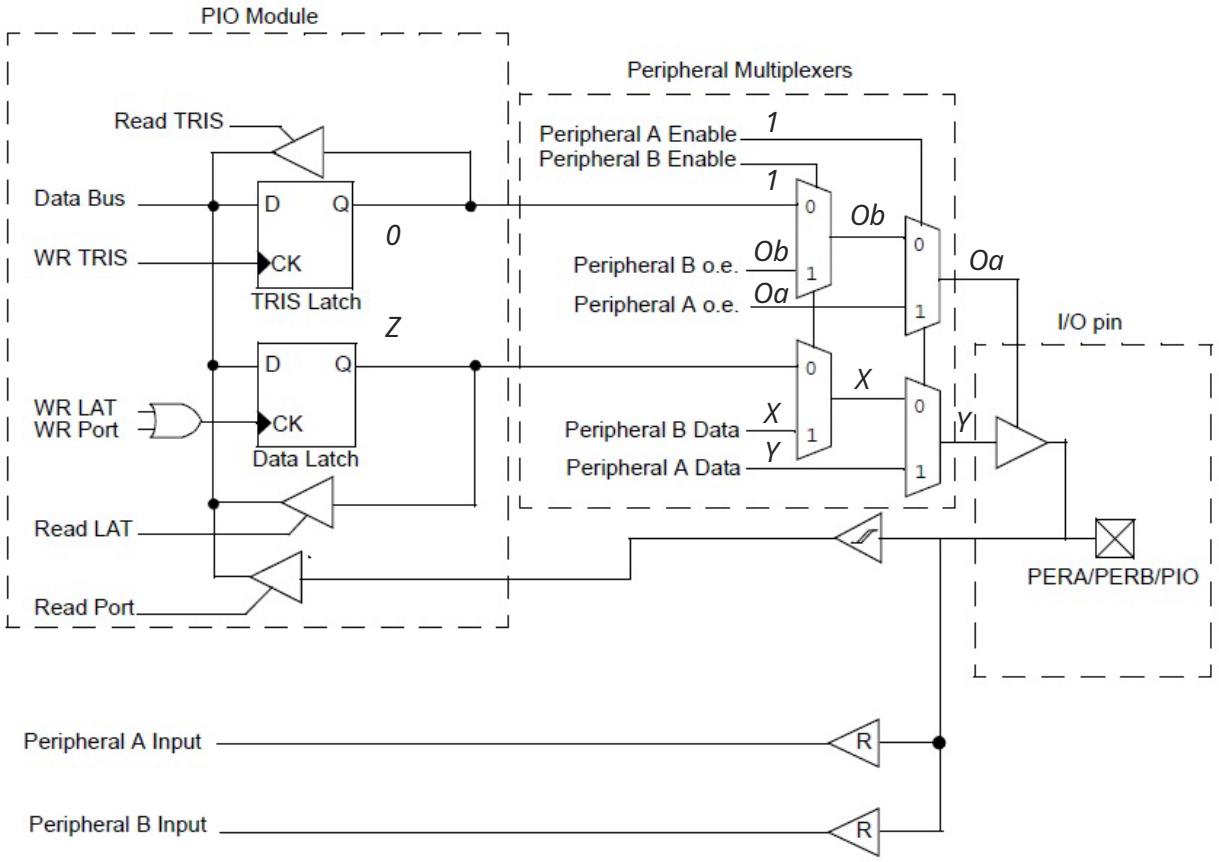
Multiplexed I/O Pins

Peripheral B is enabled, so it takes precedence w.r.t the Digital I/O



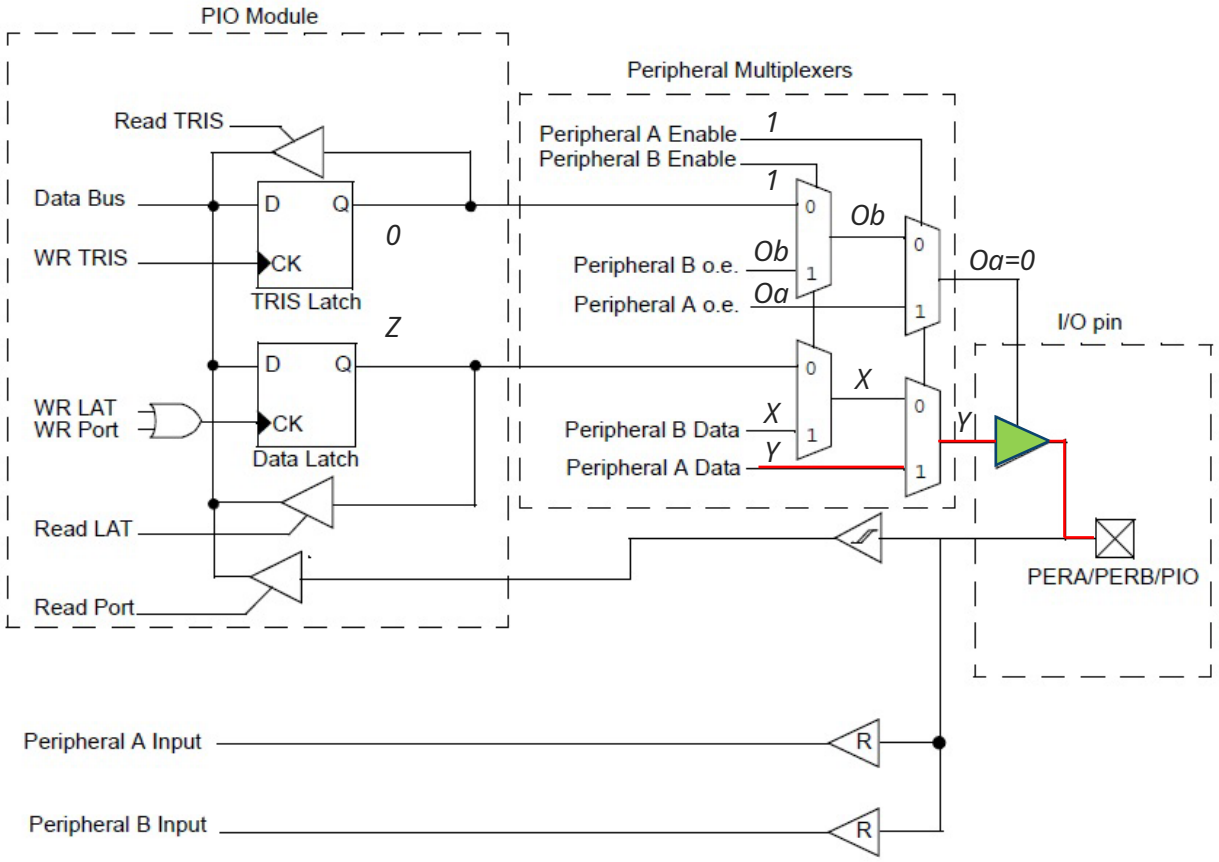
Multiplexed I/O Pins

Since Peripheral A is also enabled, its values actually make to the tristate buffer



Multiplexed I/O Pins

If Oa = 0, then Y will be on the pin



Digital I/O Ports Registers

- TRISx is the register used to select which pins which pins of a
- register are to be used as input and which as output;
- PORTx is the register used to read the voltage value of the pins;
- LATx is register is used to write a logic value as pins output;

A read of the port latch register returns the settings of the output drivers, whilst a read of the port register returns the logic levels seen on the pins.

The ’x’ stands for any of the available I/O Registers: A,B,C,D,E,F.

So we have TRISA register and TRISAbits bit field data structure, and

so on.

Bitfields Structures

Bit 15 (MSB) *Bit 0 (LSB)*

A 15	A 14				A 10	A 9		A 7	A 6	A 5	A 4	A 3	A 2	A 1	A 0
X	X				X	X		X	X	X	X	X	X	X	X

```
typedef struct tagTRISABITS {
uint16_t TRISA0:  1;
uint16_t TRISA1:  1;
uint16_t TRISA2:  1;
uint16_t TRISA3:  1;
uint16_t TRISA4:  1;
uint16_t TRISA5:  1;
uint16_t TRISA6:  1;
uint16_t TRISA7:  1;
uint16_t :1;
uint16_t TRISA9:  1;
uint16_t TRISA10: 1;
uint16_t :3;
uint16_t TRISA14: 1;
uint16_t TRISA15: 1;} TRISABITS;
extern volatile TRISABITS TRISAbits;
```

TRISA0 is a field of the structure that represents 1 bit of data

Bitfields Structures

Bit 15 (MSB)															*Bit 0 (LSB)*
A 15	**A 14**				**A 10**	**A 9**		**A 7**	**A 6**	**A 5**	**A 4**	**A 3**	**A 2**	**A 1**	**A 0**
X	X				X	X		X	X	X	X	X	X	X	1

```
typedef struct tagTRISABITS {
uint16_t TRISA0:  1;
uint16_t TRISA1:  1;
uint16_t TRISA2:  1;
uint16_t TRISA3:  1;
uint16_t TRISA4:  1;
uint16_t TRISA5:  1;
uint16_t TRISA6:  1;
uint16_t TRISA7:  1;
uint16_t :1;
uint16_t TRISA9:  1;
uint16_t TRISA10: 1;
uint16_t :3;
uint16_t TRISA14: 1;
uint16_t TRISA15: 1;} TRISABITS;
extern volatile TRISABITS TRISAbits;
```

If we write

```
TRISAbits.TRISA0 = 1;
```

Bitfields Structures

Bit 15 (MSB)															*Bit 0 (LSB)*
A 15	**A 14**				**A 10**	**A 9**		**A 7**	**A 6**	**A 5**	**A 4**	**A 3**	**A 2**	**A 1**	**A 0**
X	X				X	X		X	X	X	X	X	X	X	1

```
typedef struct tagTRISABITS {
uint16_t TRISA0:  1;
uint16_t TRISA1:  1;
uint16_t TRISA2:  1;
uint16_t TRISA3:  1;
uint16_t TRISA4:  1;
uint16_t TRISA5:  1;
uint16_t TRISA6:  1;
uint16_t TRISA7:  1;
uint16_t :1;
uint16_t TRISA9:  1;
uint16_t TRISA10: 1;
uint16_t :3;
uint16_t TRISA14: 1;
uint16_t TRISA15: 1;} TRISABITS;
extern volatile TRISABITS TRISAbits;
```

We can’t write to A8, it is not defined

Bitfields Structures

Bit 15 (MSB)															*Bit 0 (LSB)*
A 15	**A 14**				**A 10**	**A 9**		**A 7**	**A 6**	**A 5**	**A 4**	**A 3**	**A 2**	**A 1**	**A 0**
X	X				X	X		X	X	X	X	X	X	X	1

```
typedef struct tagTRISABITS {
uint16_t TRISA0:  1;
uint16_t TRISA1:  1;
uint16_t TRISA2:  1;
uint16_t TRISA3:  1;
uint16_t TRISA4:  1;
uint16_t TRISA5:  1;
uint16_t TRISA6:  1;
uint16_t TRISA7:  1;
uint16_t :1;
uint16_t TRISA9:  1;
uint16_t TRISA10: 1;
uint16_t :3;
uint16_t TRISA14: 1;
uint16_t TRISA15: 1;} TRISABITS;
extern volatile TRISABITS TRISAbits;
```

If we write

```
TRISAbits.TRISA4 = 2;
```

Bit 1	Bit 0
1	0

Bitfields Structures

Bit 15 (MSB) *Bit 0 (LSB)*

A 15	A 14				A 10	A 9		A 7	A 6	A 5	A 4	A 3	A 2	A 1	A 0
X	X				X	X		X	X	X	0	X	X	X	1

```
typedef struct tagTRISABITS {
uint16_t TRISA0:  1;
uint16_t TRISA1:  1;
uint16_t TRISA2:  1;
uint16_t TRISA3:  1;
uint16_t TRISA4:  1;
uint16_t TRISA5:  1;
uint16_t TRISA6:  1;
uint16_t TRISA7:  1;
uint16_t :1;
uint16_t TRISA9:  1;
uint16_t TRISA10: 1;
uint16_t :3;
uint16_t TRISA14: 1;
uint16_t TRISA15: 1;} TRISABITS;
extern volatile TRISABITS TRISAbits;
```

It doesn't fit -> **overflow**

```
TRISAbits.TRISA4 = 2;
```

Bit 1	Bit 0
1	0

Bitfields Structures

Bit 15 (MSB)															*Bit 0 (LSB)*
A 15	**A 14**				**A 10**	**A 9**		**A 7**	**A 6**	**A 5**	**A 4**	**A 3**	**A 2**	**A 1**	**A 0**
X	X				X	X		X	X	X	0	X	X	X	1

```
typedef struct tagTRISABITS {
uint16_t TRISA0:  1;
uint16_t TRISA1:  1;
uint16_t TRISA2:  1;
uint16_t TRISA3:  1;
uint16_t TRISA4:  1;
uint16_t TRISA5:  1;
uint16_t TRISA6:  1;
uint16_t TRISA7:  1;
uint16_t :1;
uint16_t TRISA9:  1;
uint16_t TRISA10: 1;
uint16_t :3;
uint16_t TRISA14: 1;
uint16_t TRISA15: 1;} TRISABITS;
extern volatile TRISABITS TRISAbits;
```

If we write

```
TRISA = 1;
```

Bitfields Structures

Bit 15 (MSB)															*Bit 0 (LSB)*
A 15	**A 14**				**A 10**	**A 9**		**A 7**	**A 6**	**A 5**	**A 4**	**A 3**	**A 2**	**A 1**	**A 0**
0	0				0	0		0	0	0	0	0	0	0	1

```
typedef struct tagTRISABITS {
uint16_t TRISA0:  1;
uint16_t TRISA1:  1;
uint16_t TRISA2:  1;
uint16_t TRISA3:  1;
uint16_t TRISA4:  1;
uint16_t TRISA5:  1;
uint16_t TRISA6:  1;
uint16_t TRISA7:  1;
uint16_t :1;
uint16_t TRISA9:  1;
uint16_t TRISA10: 1;
uint16_t :3;
uint16_t TRISA14: 1;
uint16_t TRISA15: 1;} TRISABITS;
extern volatile TRISABITS TRISAbits;
```

Then all bits are changed to match the 16 bit binary representation of the decimal number 1

```
TRISA = 1;
```

Bitfields Structures

Bit 15 (MSB) *Bit 0 (LSB)*

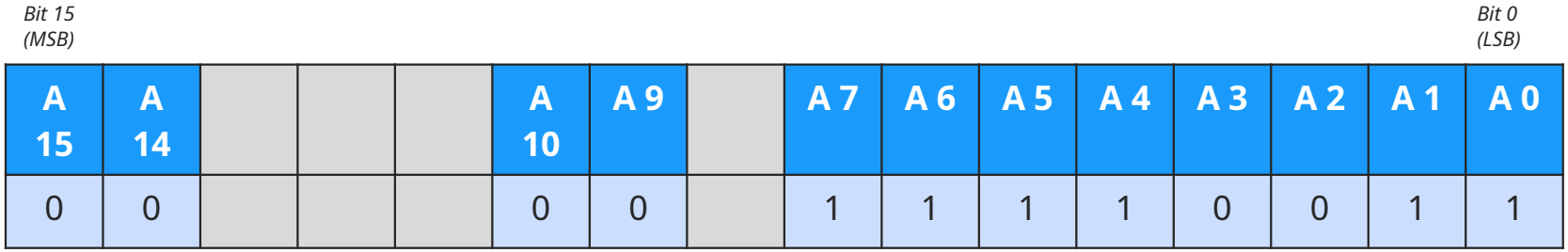
A 15	A 14				A 10	A 9		A 7	A 6	A 5	A 4	A 3	A 2	A 1	A 0
0	0				0	0		0	0	0	0	0	0	0	1

```
typedef struct tagTRISABITS {
uint16_t TRISA0:  1;
uint16_t TRISA1:  1;
uint16_t TRISA2:  1;
uint16_t TRISA3:  1;
uint16_t TRISA4:  1;
uint16_t TRISA5:  1;
uint16_t TRISA6:  1;
uint16_t TRISA7:  1;
uint16_t :1;
uint16_t TRISA9:  1;
uint16_t TRISA10: 1;
uint16_t :3;
uint16_t TRISA14: 1;
uint16_t TRISA15: 1;} TRISABITS;
extern volatile TRISABITS TRISAbits;
```

If we write

```
TRISA = 0x00F3;
```

Bitfields Structures




```
typedef struct tagTRISABITS {
uint16_t TRISA0:  1;
uint16_t TRISA1:  1;
uint16_t TRISA2:  1;
uint16_t TRISA3:  1;
uint16_t TRISA4:  1;
uint16_t TRISA5:  1;
uint16_t TRISA6:  1;
uint16_t TRISA7:  1;
uint16_t :1;
uint16_t TRISA9:  1;
uint16_t TRISA10: 1;
uint16_t :3;
uint16_t TRISA14: 1;
uint16_t TRISA15: 1;} TRISABITS;
extern volatile TRISABITS TRISAbits;
```

If we write

```
TRISA = 0x00F3;
```

Example I/O Programming

Digital input example

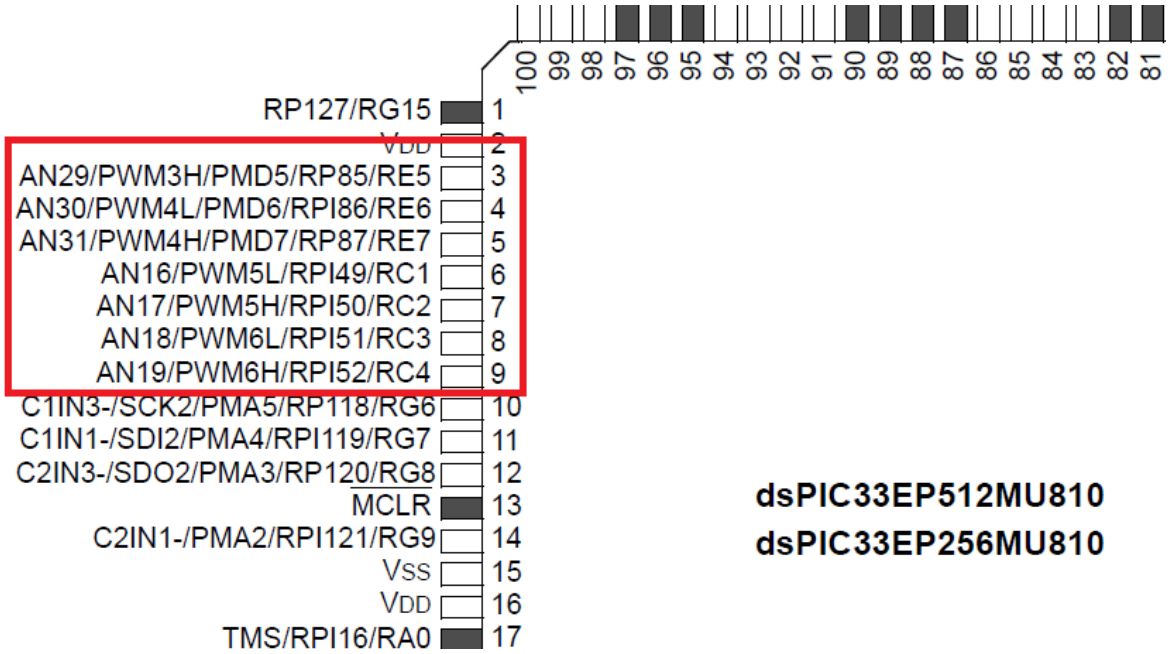
```
// set the pin as input
TRISDbits.TRISD3 = 1;
// read from pin
pinValue =
PORTDbits.RD3;
```

Digital output example

```
// set the pin as output
TRISBbits.TRISB2 = 0;
// set the pin high
LATBbits.LATB2 = 1;
```

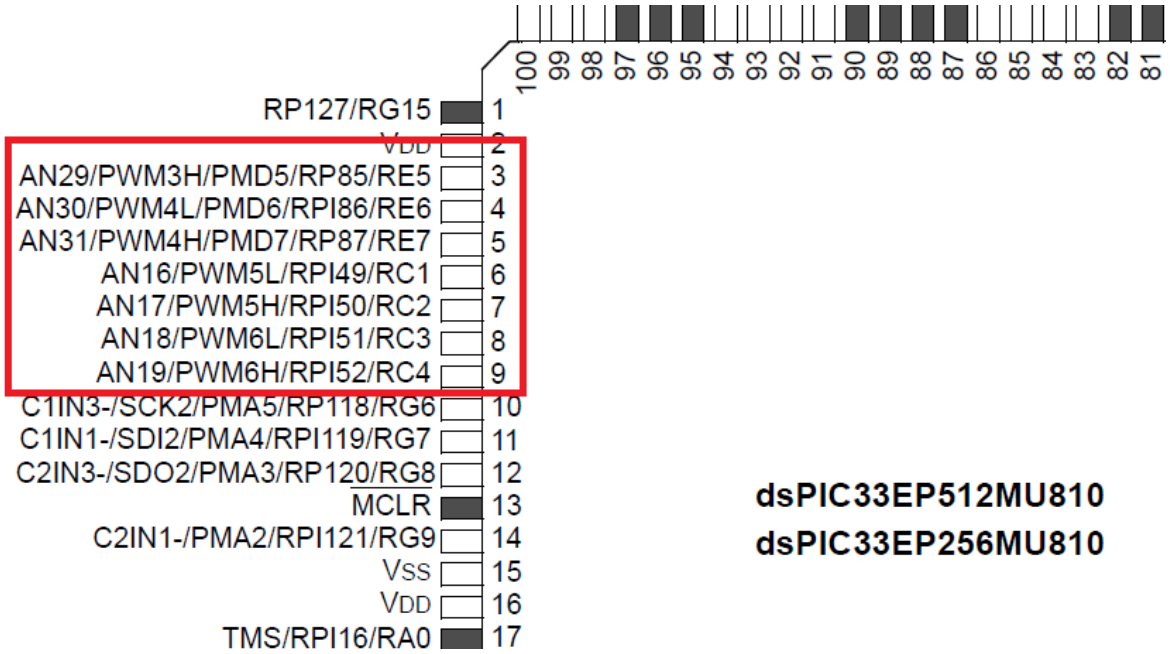
Analog & Digital Pins

Some pins are both analog and digital



Analog & Digital Pins

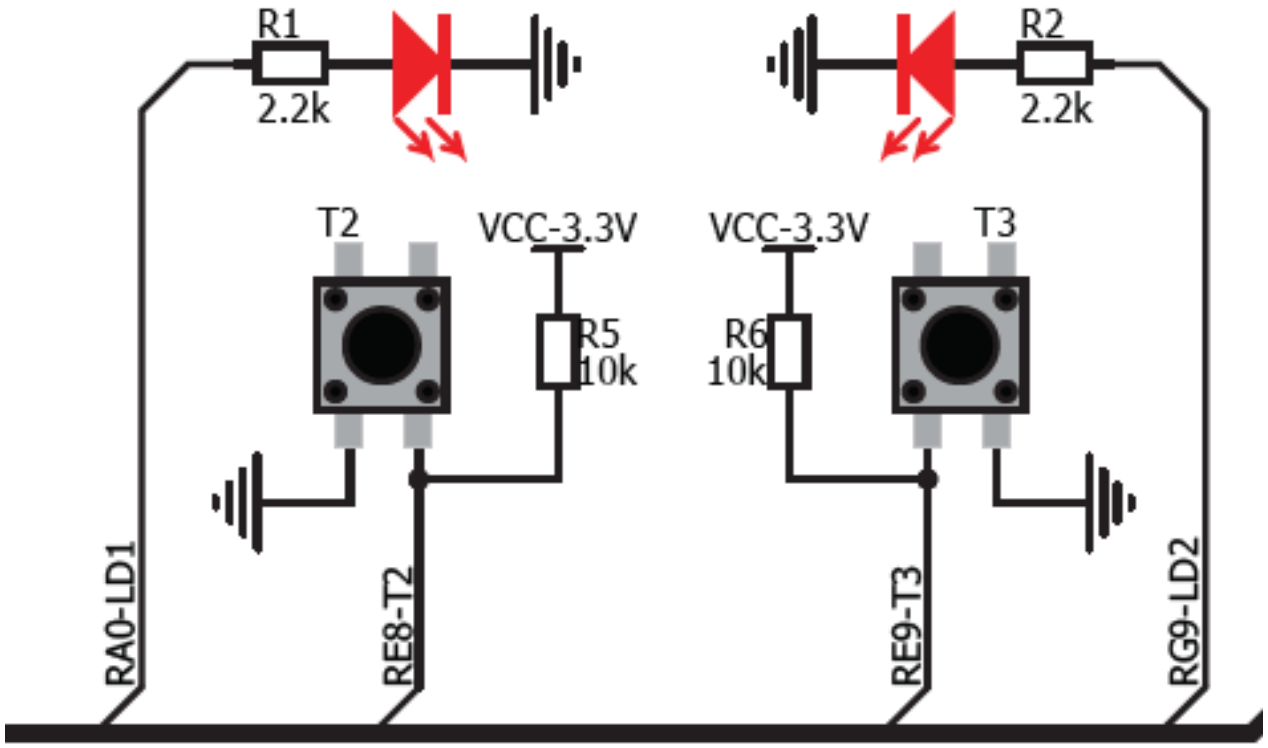
Some pins are both analog and digital -> **default is analog!**



Making Ports Digital

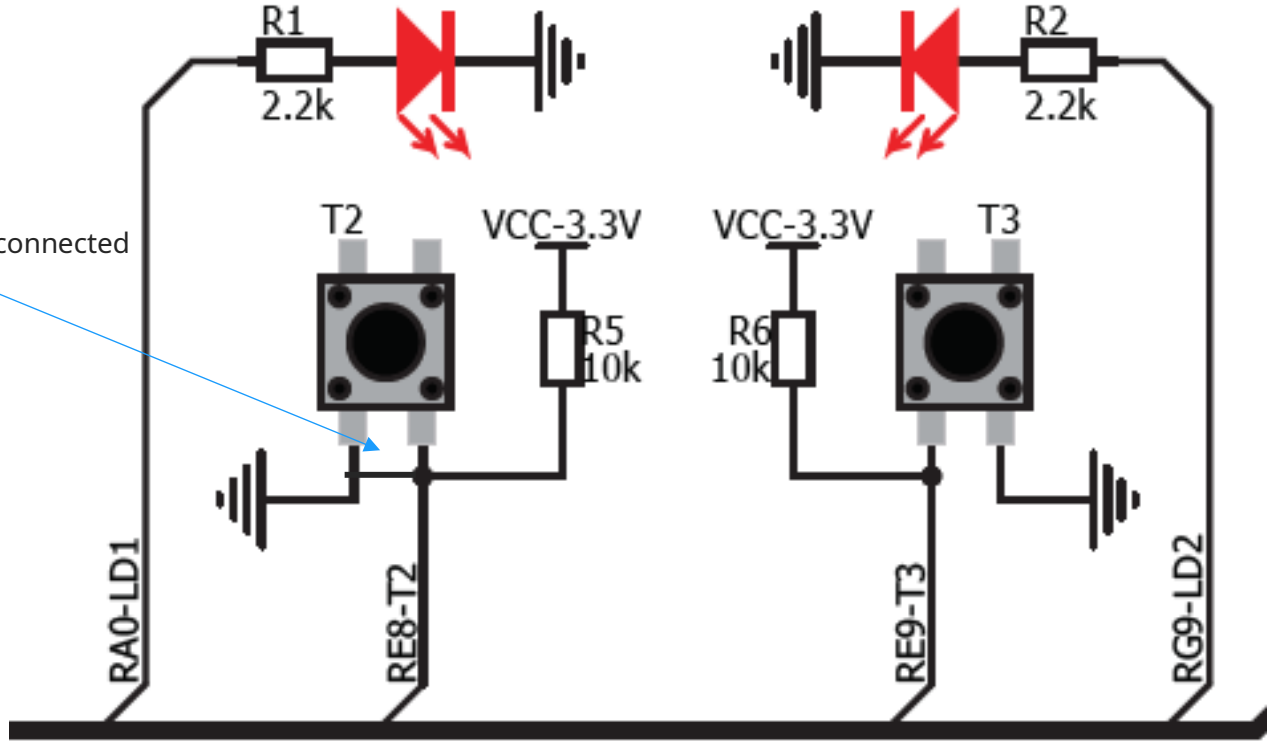
```c
#include <xc.h>
int main(void) {
  // all analog pins disabled
  ANSELA = ANSELB = ANSELC = ANSELD = ANSELE = ANSELG = 0x0000;
  return 0;
}
```

Let's Play With Buttons and LEDs



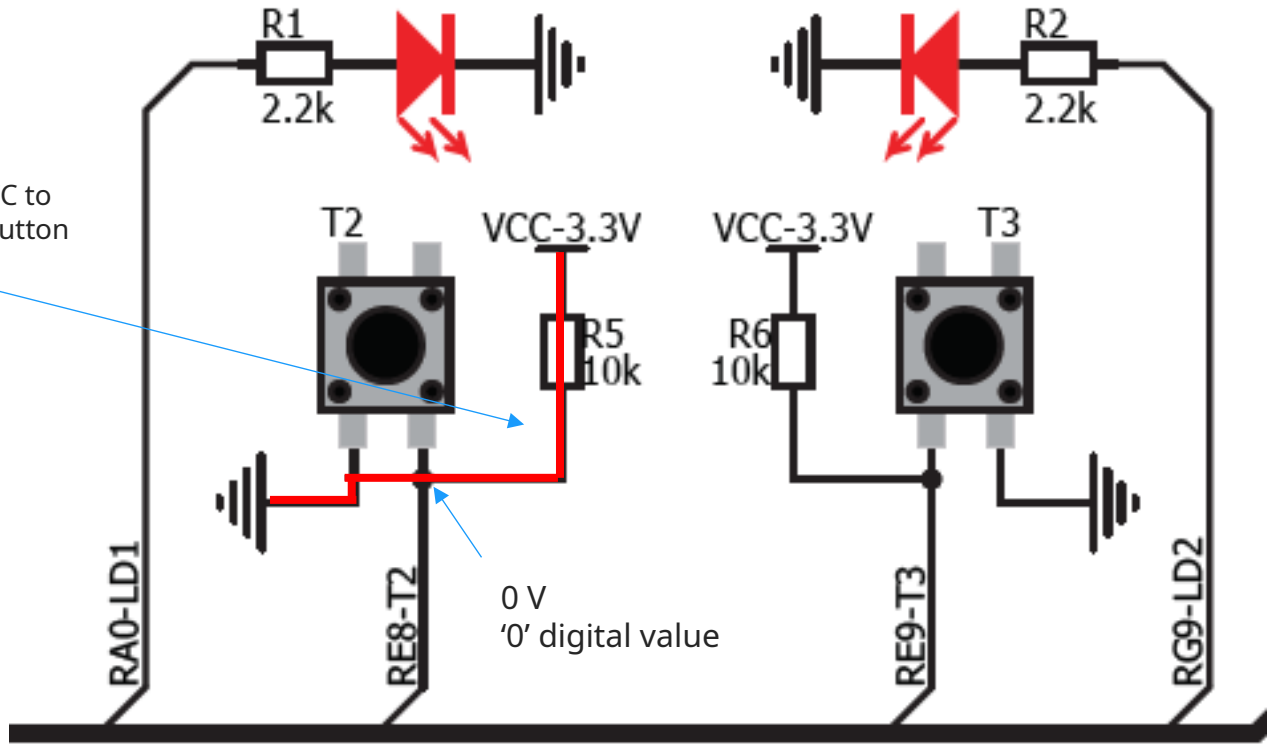
Let’s Play With Buttons and LEDs

If we press the button
the two pins become connected



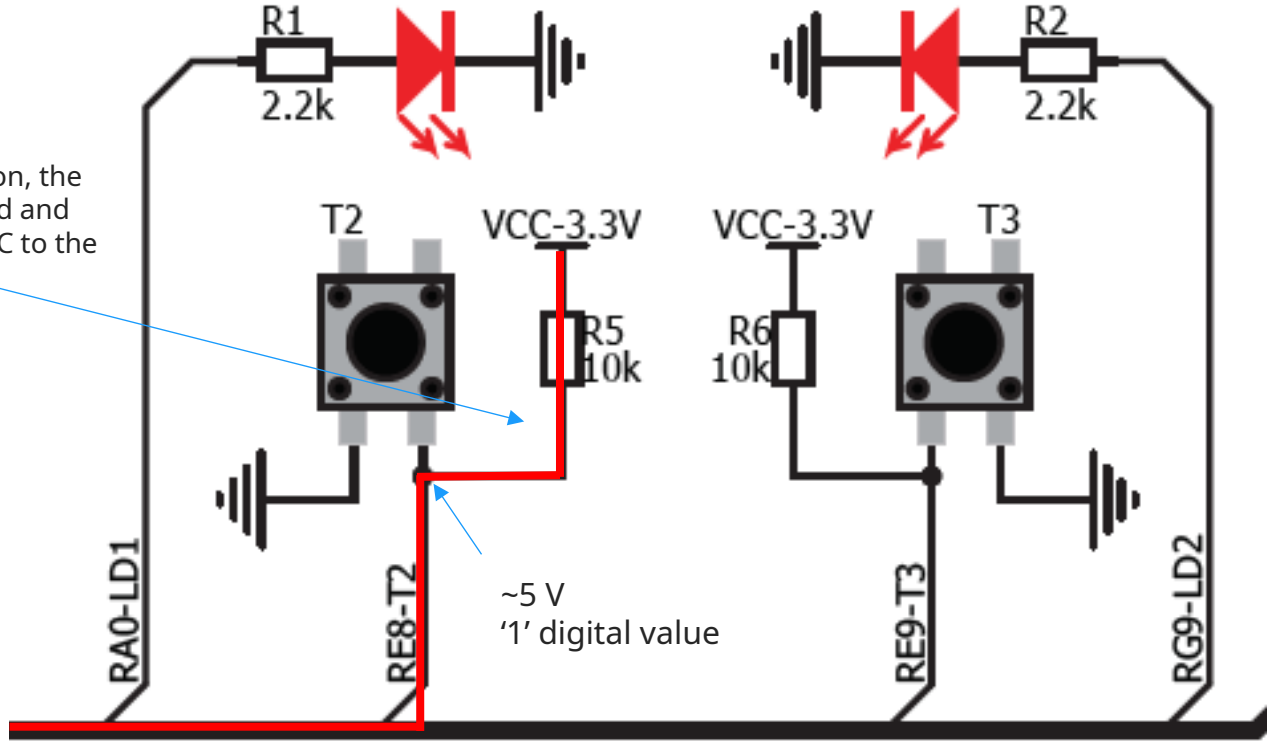
Let’s Play With Buttons and LEDs

Current flows from VCC to ground through the button pins



Let’s Play With Buttons and LEDs

If we release the button, the pins are not connected and current flows from VCC to the microcontroller

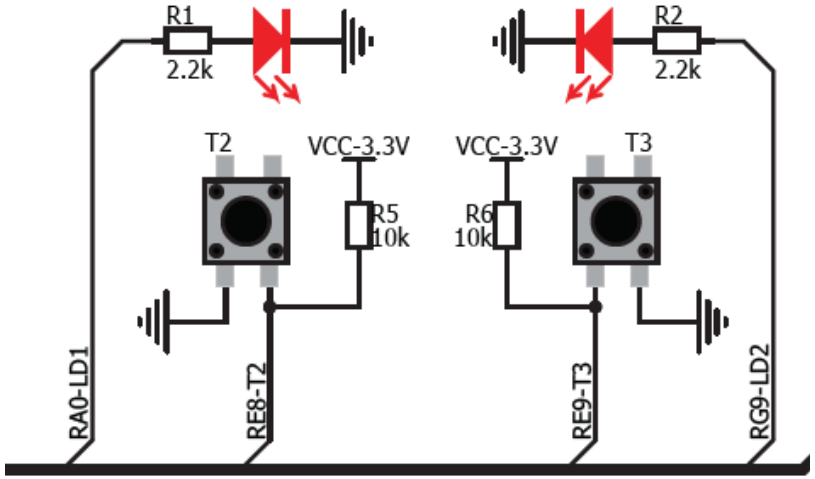


Assignment

1/ Turn on the led LD1

2/ Read the button T2; while the button is pressed turn on the led LD1, otherwise turn it off.

3/ Whenever the button T2 is pressed, toggle the status of the led LD1 (once per button click).





Università
di Genova